AI URBAN MICRO MOBILITY SIMULATOR · GAP ANALYSIS

What is left to build

The repository compared with the *Detailed Project Approach* PDF, and the work needed to close each gap.

Prepared 15 October 2026 · Based on commit `358a456` on `master` · Based on reading the code. The test suite and the app were not run for this review.

~60–65%
of the approach document is in place

5 / 11
deliverables fully done

7
tasks left (1 critical, 3 high)

6–8 days
rough estimate for one developer

The one that matters most. The approach document says the AI component is a model that **predicts traffic**, and that prediction must feed route scoring (§6, §18). The model in the repository predicts **rider demand**. Routing takes congestion straight from the stored traffic snapshots, and no model touches it. As things stand, an examiner could fairly call the router "A* with an AI label", which the document warns against. Task 1 fixes this. Do it first.

1. Status of each deliverable

Deliverables as listed in §17 of the approach document.

#	Deliverable	Status	What exists today
1	Python backend for routing and simulation	DONE	FastAPI service in `apps/ml`
2	Dijkstra baseline, optional A*	HALF	A* only (`routing/astar.py`). No Dijkstra, no comparison of the two
3	Synthetic road and traffic datasets	DONE	Seeded Belagavi network on real OpenStreetMap streets, plus the synthetic traffic generator
4	ML traffic-prediction model	MISSING	A LightGBM model exists, but it predicts demand, not traffic
5	Travel-time and energy modules	MOSTLY	Travel time slows with congestion. Energy is Wh per km plus climbing, with no traffic or stop factor
6	Multi-factor route optimizer	PARTIAL	Weighted cost per road segment, with 3 fixed profiles. No normalized score per route, and the weights can't be changed
7	FastAPI endpoints	DONE	Endpoints exist under different names (`/internal/routing/*`, `/internal/sim/*`)
8	Interactive Leaflet map	DONE	Leaflet with OpenStreetMap streets
9	Simulation controls and comparison dashboard	PARTIAL	Plan screen compares 3 routes. The simulation runs for the whole fleet; no single e-bike is animated and there are no start/pause/reset buttons

#	Deliverable	Status	What exists today
10	Evaluation results for several scenarios	PARTIAL	Fleet scenarios can be compared. The 6 test scenarios in §13 have not been run and recorded
11	Report, diagrams, test cases, presentation	DONE	Final report, user manual and viva guide, plus 41 test files. These need updating after the tasks below

2. Recommended order

The order follows the approach document's own rule (§15): baseline first, then the AI layer and optimization, then simulation, then evaluation.

#	Task	Priority	Rough effort
1	Traffic-prediction model, wired into routing	CRITICAL	1.5–2 days
2	Dijkstra baseline and comparison with A*	HIGH	2–3 hours
3	Normalized score per route, with weights you can adjust	HIGH	1 day
4	Step-by-step simulation of one e-bike, animated on the map	HIGH	1.5–2 days
5	Add traffic and stop factors to the energy model	MEDIUM	Half a day
6	Run and record the 6 evaluation scenarios	MEDIUM	1 day
7	Update the report, viva guide and user manual	MEDIUM	Half a day

> The effort figures are rough estimates, not measurements. They assume someone who already knows this codebase.

3. Tasks in detail

01 Traffic-prediction model, wired into routing CRITICAL 1.5–2 days

THE APPROACH DOCUMENT ASKS FOR

A model trained on traffic history, using the features in §6: hour, day of week, road id, distance, road type and previous traffic. Its prediction is the traffic value used for travel time and route scoring.

THE REPOSITORY HAS

`models/demand.py` predicts rider demand per zone. Routing reads `COALESCE(ts.congestion_index, 0)` from `traffic_snapshots` (`routes/service.ts`), so no prediction is involved.

WORK

1. Add `apps/ml/app/models/traffic.py` . Copy the demand model's structure: LightGBM, chronological train/test split, MAE/RMSE/MAPE against a seasonal-naive baseline, and an entry in `model_registry` . The training data is the existing synthetic traffic history.
2. Features: `hour` , `day_of_week` , `segment_id` (categorical), `length_m` , `road_class` , and lagged congestion for the previous slot and the same slot yesterday. Target: `congestion_index` .
3. Add `POST /internal/ml/predict/traffic` . It takes an hour and a day and returns predicted congestion for every segment in one batch. Cache the result through `cache/index.ts` (e.g. `traffic:pred:<day>:<hour>`). Training runs as a job, never inside a request.
4. Give route planning a **traffic source** option: *live snapshot* (today's behaviour) or *AI prediction for a chosen time*. With the second option, the predicted congestion replaces the snapshot value on every edge before A* runs.
5. On the Plan screen, add a time-of-day control (morning, afternoon, evening, peak) and a traffic-scenario control (low, medium, high or predicted).

DONE WHEN

- The model is recorded in `model_registry` with its metrics and beats the seasonal-naive baseline
- Planning the same trip for 03:00 and for evening peak gives different congestion, and on at least one trip a different route
- A pytest test asserts that predictions are in the range 0–1 and that training is reproducible with a fixed seed

02 Dijkstra baseline and comparison with A* HIGH

2–3 hours

THE APPROACH DOCUMENT ASKS FOR

Dijkstra as the traditional baseline, built with the 7 steps in §5. A* comes after it as an optional improvement, and the two are compared on the same graph.

THE REPOSITORY HAS

Only A*, through `nx.astar_path`. The word "dijkstra" does not appear anywhere in the code.

WORK

1. Write `apps/ml/app/routing/dijkstra.py` by hand with `heapq`, following the document's steps: initial distances, a priority queue, edge relaxation and a predecessor map. A hand-written version is easier to explain in the viva than a library call. Count how many nodes it settles.
2. Add an `algorithm` parameter (`dijkstra` or `astar`) to the plan endpoint, and return `computeMs` and `nodesExpanded` with every route.
3. Show both on the Plan screen: same route and cost, different compute time and number of nodes explored.

DONE WHEN

- A test asserts that hand-written Dijkstra, `nx.dijkstra_path` and A* give the same total cost on a set of random origin–destination pairs
- A* expands fewer nodes than Dijkstra on long trips

03 Normalized score per route, with weights you can adjust HIGH

1 day

THE APPROACH DOCUMENT ASKS FOR

Generate several candidate routes, then compute each route's metrics and normalize them. Score = 0.40·Distance + 0.30·Time + 0.20·Traffic + 0.10·Energy, and the lowest score wins. Keep the other routes for comparison, and support experiments that change the weights (§8, §13 scenario 6).

THE REPOSITORY HAS

Weights apply to each road segment's cost inside A* (`costs.py`). There are 3 fixed profiles, and each produces one route. Nothing scores whole routes against each other or picks a winner that way.

WORK

1. Add `apps/ml/app/optimization/route_optimizer.py`. Collect candidates from the 3 existing profiles plus the pure shortest-distance route. Optionally add k-shortest paths (`nx.shortest_simple_paths`), with k capped so a response stays under 300 ms. Remove duplicate routes.
2. Scale each metric to 0–1 with min–max across the candidates (if every candidate has the same value, use 0). Apply the weights and pick the lowest score.
3. Add `POST /internal/routing/optimize`. It takes the weights and returns every candidate with its raw metrics, normalized metrics and score, plus the id of the winner.
4. Plan screen: weight sliders (kept summing to 1) and an *AI-optimized* badge on the winning route. Show the other candidates as thinner or greyed-out lines on the map, and add a score column to the comparison table.

DONE WHEN

- A test with three hand-built candidates reproduces the worked example in §16: route B wins even though it isn't the shortest
- Moving the weight sliders changes the winning route on at least one trip in the demo

04 Step-by-step simulation of one e-bike, animated on the map HIGH 1.5–2 days

THE APPROACH DOCUMENT ASKS FOR

One e-bike moving along the chosen route, segment by segment. Its state is `current_node`, `current_edge`, distance travelled, elapsed time, energy used, traffic level and status. The screen shows start/pause/reset, a live metric panel and a comparison chart at the end (§9, §10).

THE REPOSITORY HAS

A SimPy engine that models the whole fleet moving between zones (`sim/engine.py`, which describes itself as working at zone level). The web app has no animation code.

WORK

1. Add `apps/ml/app/sim/journey.py`. Given a route's steps, it walks them at a fixed time step (for example 1 second) and outputs a trace. Each frame records position, the document's state variables, and the effective speed for that segment. Keep this logic on the server; React only plays the trace back.
2. Add `POST /internal/sim/journey`, plus an API route that saves each run to a new `route_simulations` table with the fields from §12: source, destination, scenario, selected route, distance, time, energy, traffic, score and created time. Add a matching `GET` for past results.
3. Plan screen: an e-bike marker that moves along the route, start/pause/reset and playback-speed buttons, and a live panel showing distance, elapsed time, current traffic and energy.
4. When the run finishes, show a bar chart (Recharts) comparing the chosen route with the other candidates on distance, time, traffic and energy.

DONE WHEN

- The trace's final totals match the route's distance, time and energy (a pytest test checks this)
- Pause, resume and reset work, and a saved run can be reopened and played again

05 Add traffic and stop factors to the energy model MEDIUM Half a day

Today the energy figure is Wh per km plus a climbing term (`astar.py`). The document's model (§7.2) is Energy ≈ Distance × BaseConsumption × TrafficFactor × SlopeFactor × StopFactor. Move the energy calculation into `apps/ml/app/sim/energy.py` and add two factors: a traffic factor that grows with congestion (stop-and-go riding) and a stop factor based on the number of junctions on the route (nodes with three or more roads meeting). Keep the coefficients as named, documented constants, since the document calls them "simulation parameters".

DONE WHEN

- Two routes of similar length but different congestion or junction counts give clearly different energy figures, and a test checks this

06 Run and record the 6 evaluation scenarios MEDIUM 1 day

Write a script (for example `scripts/evaluate-routing.mjs`, or a pytest module) that runs each scenario from §13 on fixed trips and writes the results to `docs/evaluation-results.md`. Each result records distance, time, traffic exposure, energy, score and **computation time**.

#	Scenario	Expected behaviour
1	Low traffic	The shortest route wins or is close to winning
2	Heavy traffic on the shortest route	The optimizer picks a different route
3	Steep route	The energy term counts against it
4	Peak hour (predicted traffic)	The selection differs from the off-peak selection
5	Low-energy weighting	The route with the lowest energy wins
6	Changing the weights	A table showing which route wins as the weights change

DONE WHEN

- All 6 scenarios run with one command and give the same results on every run (fixed seed)
- The results table is in the repo, ready to drop into the report

07 Update the report, viva guide and user manual MEDIUM Half a day

- **Report:** add the traffic model's metrics, the Dijkstra vs A* comparison, the scoring formula, the evaluation table and screenshots of the animated journey.
- **Viva guide:** rewrite the answer to "Where is AI used?" around the traffic prediction feeding the route score (§18), and add a demo step that changes the time of day and shows the route change.
- **User manual:** document the new Plan-screen controls (traffic source, time of day, weight sliders, simulation playback).
- `MODULES.md`: add the new endpoint contracts. Write an ADR on why routing scores whole routes, and how that sits alongside the per-segment profile weights.

4. Built beyond the approach document

§1 of the document excludes user accounts, payments, booking, rental, station management and fleet management. The repository includes accounts and roles, a vehicle registry, fleet rebalancing by linear programming, charging stations, live traffic over sockets, weather, analytics and a GBFS feed. **Keep all of it.** It works, and removing it would only add risk. Present it in the viva as the platform the simulator runs on, and keep the demo focused on the e-bike route-optimization workflow, because that is what the document grades.

The stack also differs from the document's suggestion (React, Express and MySQL instead of plain HTML/JS and SQLite). Be ready to explain why in one sentence: the project is built to grow past a single-user prototype.